BATCH 29

AI - 13.5

Task Description #1 (Refactoring – Removing Global Variables)
• Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
• Expected Output:
o Refactored version passing rate as a parameter or using a configuration structure.

Prompt

Refactor the following Python code to eliminate the use of global variables. Pass required values as function parameters or store them in a configuration structure to improve modularity and testability.

ai13.5

ai13.5 > ...

```python
#refactoring code to make it more readable and maintainable and removing global variables
#Sample Legacy Code:
#rate = 0.1
#def calculate_interest(amount):
#return amount * rate
#print(calculate_interest(1000))
#Refactored Code:
class InterestCalculator:
    def __init__(self, rate):
        self.rate = rate

    def calculate_interest(self, amount):
        return amount * self.rate
interest_calculator = InterestCalculator(0.1)
print(interest_calculator.calculate_interest(1000))
```

JUSTIFICATION

- Removes dependency on **global state**.
- Functions become **pure and reusable**.
- Easier to **test** with different interest rates.
- Improves **modularity** since functions no longer rely on external variables.

Task Description #2 : (Refactoring Deeply Nested Conditionals)
• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas:
o Readability
o Logical simplification
o Maintainability
Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:
o Flattened logic using guard clauses or a mapping-based approach.

Code:

ai13.5

ai13.5 > ...

```python

#refactoring code to improve performance and reduce memory usage
#Legacy Code:
#score = 78
#if score >= 90:
#print("Excellent")
#else:
#if score >= 75:
#print("Very Good")
#else:
#if score >= 60:
#print("Good")
#else:
#print("Needs Improvement")
#Refactored Code:
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

Justification

- Removes **deep nesting**.
- Improves **code readability**.
- Easier to maintain and modify grading logic.
- Mapping approach allows **extensibility**.

Task 3 (Refactoring Repeated File Handling Code)
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers
o Function reuse
Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()

f = open("data2.txt")
print(f.read())
f.close()

Code:

```python
#refactoring repeated code into functions to improve readability and maintainability
#Legacy Code:
#f = open("data1.txt")
#print(f.read())
#f.close()
#f = open("data2.txt")
#print(f.read())
#f.close()
#Refactored Code:
def read_and_print_file(filename):
    with open(filename) as f:
        print(f.read())
read_and_print_file("data1.txt")
read_and_print_file("data2.txt")
```

Justification
Applies **DRY (Don't Repeat Yourself)** principle.

Uses **context manager (`with open`)** to automatically close files.

Improves **reusability**.

Reduces risk of **file descriptor leaks**.

Task 4 (Optimizing Search Logic)
• Task: Refactor inefficient linear searches using appropriate data structures.
• Focus Areas:
o Time complexity
o Data structure choice
Legacy Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:

if u == name:
found = True
print("Access Granted" if found else "Access Denied")
Expected Outcome:
o Use of sets or dictionaries with complexity justification.

Code

```
#optimizing search algorithm from O(n) to O(log n) using binary search
#Legacy Code:
#users = ["admin", "guest", "editor", "viewer"]
#name = input("Enter username: ")
#found = False
#for u in users:
#if u == name:
#found = True
#print("Access Granted" if found else "Access Denied")
#Refactored Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Justification

- Lists require **O(n)** search.
- Sets provide **O(1) average lookup time**.
- Code becomes **simpler and faster**.
- Eliminates unnecessary loops and flags.

Task 5 (Refactoring Procedural Code into OOP Design)
• Task: Use AI to refactor procedural code into a class-based design.
• Focus Areas:
o Object-Oriented principles
o Encapsulation
Legacy Code:

salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)

Code

```
#refactoring code to use list comprehensions for better readability and performance
#Legacy Code:
#salary = 50000
#tax = salary * 0.2
#net = salary - tax
#print(net)
#Refactored Code:
salary = 50000
net = salary * (1 - 0.2)
print(net)
```

Justification

- Implements **encapsulation**.
- Salary logic is grouped inside a **class**.
- Easier to extend for **bonuses, deductions, etc.**
- Promotes **object-oriented design principles**.

Task 6 (Refactoring for Performance Optimization)
• Task: Use AI to refactor a performance-heavy loop handling large data.
• Focus Areas:
o Algorithmic optimization
o Use of built-in functions
Legacy Code:
total = 0
for i in range(1, 1000000):
if i % 2 == 0:
total += i
print(total)
Expected Outcome:
o Optimized logic using mathematical formulas or

comprehensions, with time comparison.

Code

```
#Refactoring for Performance Optimization
#Legacy Code:
#total = 0
#for i in range(1, 1000000):
#if i % 2 == 0:
#total += i
#print(total)
#Refactored Code:
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
print(total)
```

Justification

- Original loop runs **1 million iterations**.
- `range(2, 1000000, 2)` reduces iterations by half.
- Mathematical formula gives **O(1) complexity**.
- Significantly improves **performance**.

Task 7 (Removing Hidden Side Effects)
• Task: Refactor code that modifies shared mutable state.
• Focus Areas:
o Functional-style refactoring
o Predictability
Legacy Code:
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
Expected Outcome:

o Refactored function returning values instead of mutating globals.

Code

```
#removing hidden side effects and improving code clarity
#Legacy Code:
#data = []
#def add_item(x):
#data.append(x)
#add_item(10)
#add_item(20)
#print(data)
#Refactored Code:
class ItemManager:
    def __init__(self):
        self.data = []

    def add_item(self, x):
        self.data.append(x)
item_manager = ItemManager()
item_manager.add_item(10)
item_manager.add_item(20)
print(item_manager.data)
```

Justification
Avoids modifying **global mutable state**.

Function becomes **predictable and pure**.

Easier to **debug and test**.

Encourages **functional programming style**.

Task 8 (Refactoring Complex Input Validation Logic)
• Task: Use AI to simplify and modularize complex validation rules.
• Focus Areas:
o Readability
o Testability

Legacy Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

Code

```python
#(Refactoring Complex Input Validation Logic)
#Legacy Code:
#password = input("Enter password: ")
#if len(password) >= 8:
#if any(c.isdigit() for c in password):
#if any(c.isupper() for c in password):
#print("Valid Password")
#else:
#print("Must contain uppercase")
#else:
#print("Must contain digit")
#else:
#print("Password too short")
#Refactored Code:
def validate_password(password):
    if len(password) < 8:
        return "Password too short"
    if not any(c.isdigit() for c in password):
        return "Must contain digit"
    if not any(c.isupper() for c in password):
        return "Must contain uppercase"
    return "Valid Password"
password = input("Enter password: ")
print(validate_password(password))
```

Justification

- Breaks complex logic into **small reusable functions**.
- Improves **readability and maintainability**.
- Easier to **unit test each validation rule**.
- Reduces **nested conditionals**.